MUHAMMAD TAHA
22F-3277
BCS-3C
LAB 9

LAB 9

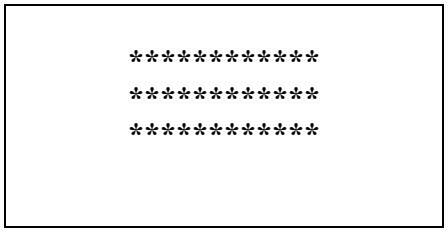
(Total Marks 15)

Instructions for today lab

- **Use Stack, Sub-Routines and Loop.**
- **You will pass parameter from start through stack. Remaining implementation should be in sub-routine.**
- **Use clear screen.**
- **Hard Code will be marked zero.**
- **Calculate its location using formula location = (y position * 30 + x position) * 3**

Task 1: Write assembly language code to display square box like following:

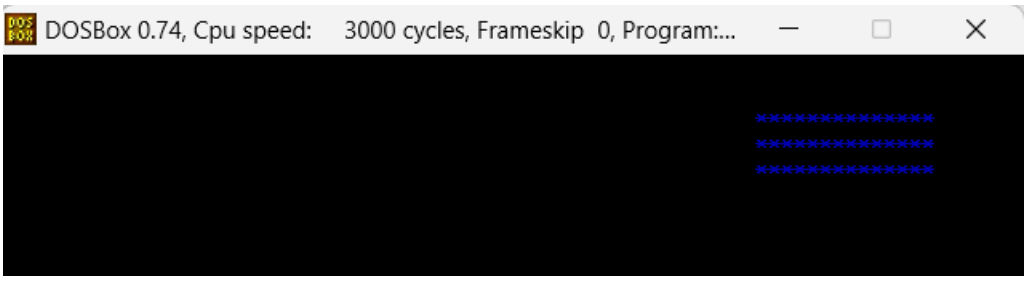
(Note: Length and width of your square box must be equal to highest digit in your roll number)




```
org 0x0100
jmp start

clrscr:
        push ax
        push di
        push es

        mov ax, 0xb800
        mov es, ax
```

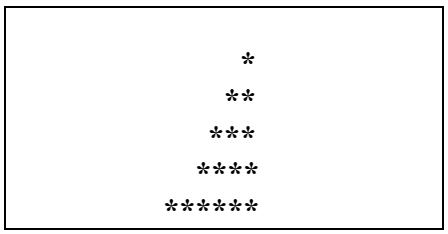
```
        mov di, 0

nextchar:
        mov word [es:di], 0x0720
        add di, 2
        cmp di, 4000
        jne nextchar

        pop es
        pop di
        pop ax
        ret

rectangle:
        push bp
        mov bp, sp
        push ax
        push bx
        push cx
        push dx
        push es
        push di

        mov ax, 30h
        mov dx, [bp+4]
        mul dx
        add ax, [bp+6]   ;ax=ax+3
        mov dx, 3
        mul dx                  ;ax= ax*dx
        mov di,ax
        mov si, di              ;copy of position

        mov ax, 0xb800
        mov es, ax


        mov bx, [bp+8]  ;height

l2:
        mov cx, [bp+10] ;length
        l1:
                mov word [es:di], 0x012A
                add di, 2
                dec cx
                cmp cx,0
                jne l1

        mov di,0
```

```
        add si,160
        mov di,si

        dec bx
        cmp bx, 0
        jne l2

        pop di
        pop es
        pop dx
        pop cx
        pop bx
        pop ax
        ret 8

start:
        call clrscr

        mov ax, 7h
        imul ax, 2h
        push ax     ;length  (+10)

        mov ax, 3h
        push ax             ;height   (+8)

        mov ax, 2h
        push ax             ;x-value (+6)
        mov ax, 3h
        push ax             ;y-value (+4)

        call rectangle

        mov ax, 0x4c00
        int 0x21
```



Task 2: Write assembly language code to display triangle like following:

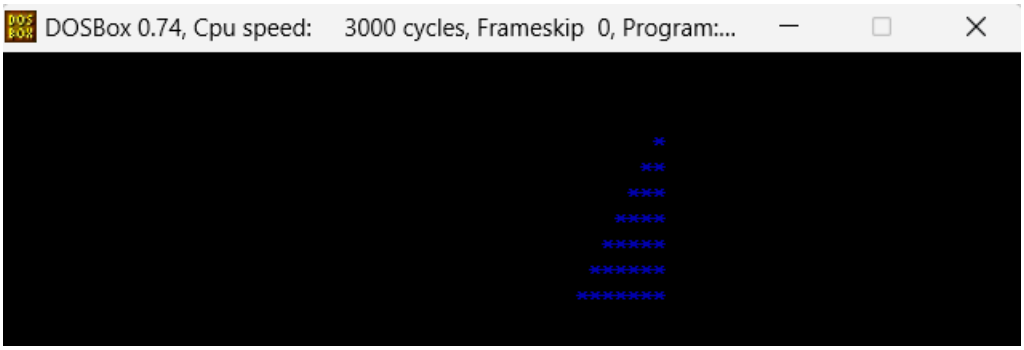
(Note: number of rows in triangle must be equal to highest digit in your roll number)




```
org 0x0100
jmp start

clrscr:
        push ax
        push di
        push es

        mov ax, 0xb800
        mov es, ax
        mov di, 0

nextchar:
        mov word [es:di], 0x0720
        add di, 2
        cmp di, 4000
        jne nextchar

        pop es
        pop di
        pop ax
        ret

triangle:
        push bp
        mov bp, sp
        push ax
        push bx
        push cx
        push dx
        push es
        push di

        mov ax, 30h
        mov dx, [bp+4]
        mul dx                  ;ax = 30*3
```

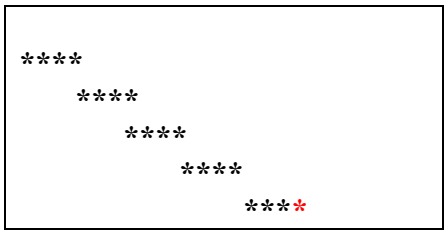
```
        add ax, [bp+6]   ;ax=ax+3
        mov dx, 3
        mul dx                  ;ax= ax*dx
        mov di, ax
        mov si, di              ; copy of di

        mov ax, 0xb800
        mov es, ax

        mov bx, [bp+8]  ;length

        mov cx, 0
        mov dx, 1
l1:

        l2:
                mov word [es:di], 0x012a
                sub di, 2
                inc cx
                cmp cx,dx
                jl l2

        mov cx, 0
        add si, 160
        mov di, si
        add dx, 1
        cmp dx, bx
        jle l1


        pop di
        pop es
        pop dx
        pop cx
        pop bx
        pop ax
        ret 6

start:
        call clrscr

        mov ax, 7h
        push ax         ;rows   (+8)

        mov ax, 2h
        push ax         ;x-value (+6)
        mov ax, 4h
        push ax         ;y-value (+4)
```

```
call triangle

mov ax, 0x4c00
int 0x21
```



Task 3: Write a code which will take position and length of snake and then print it like following:

(Note: number of * in each row must be equal to digit near to 5 in your roll number..)




```
org 0x0100
jmp start

clrscr:
        push ax
        push di
        push es

        mov ax, 0xb800
        mov es, ax
        mov di, 0

nextchar:
        mov word [es:di], 0x0720
        add di, 2
        cmp di, 4000
        jne nextchar

        pop es
        pop di
        pop ax
        ret

parallelogram:
        push bp
        mov bp, sp
        push ax
        push bx
        push cx
        push dx
        push es
        push di

        mov ax, 30h
        mov dx, [bp+4]
```

```
        mul dx
        add ax, [bp+6]   ;ax=ax+3
        mov dx, 3
        mul dx                  ;ax= ax*dx
        mov di,ax

        mov ax, 0xb800
        mov es, ax

        mov bx, [bp+8]  ;height

l2:
        mov cx, [bp+10] ;length
        l1:
                mov word [es:di], 0x012A
                add di, 2
                dec cx
                cmp cx,0
                jne l1

        add di,160

        dec bx
        cmp bx, 0
        jne l2

        pop di
        pop es
        pop dx
        pop cx
        pop bx
        pop ax
        ret 8

start:
        call clrscr

        mov ax, 7h
        push ax    ;length  (+10)

        mov ax, 5h
        push ax          ;height   (+8)

        mov ax, 0h
        push ax          ;x-value (+6)
        mov ax, 0h
        push ax          ;y-value (+4)
```

```
call parallelogram

mov ax, 0x4c00
int 0x21
```